Immix: A Mark-Region Garbage Collector with Space Efficiency, Fast Collection, and Mutator Performance

Stephen M. Blackburn, Kathryn S. McKinley

Szabó Márk

Garbage Collection Background

Why Garbage Collection Matters

Managed languages (Java, C#) allocate many **small, short-lived** objects, so the garbage collector **directly determines** performance.

Applications make conflicting demands:

- real-time: **pause-time guarantees**
- embedded: **space efficiency**
- servers: **throughput**

Conflicting Demands

Throughput and space efficiency pull the collector in opposite directions:

Demand	Requires	Costs
Throughput	copying for cache locality large heap, rare collections	2× space memory
Space efficiency	non-moving collection small heap	worse locality GC time dominates

No single winner

Locality needs moving objects, which costs space. Saving space means not moving them, which loses locality.

Tracing Garbage Collection

TODO: The three jobs of a tracing collector: allocation, identification of live objects (transitive closure from roots), reclamation of free memory. The paper only considers tracing collectors: reference counting is left out because it cannot reclaim cycles.



Three Reclamation Strategies

TODO: Sweep-to-free-list (mark-sweep), evacuation (semi-space), compaction (mark-compact). Reclamation strategy dictates allocation strategy.

TikZ diagram: the same fragmented heap after each strategy: free list holes, live objects copied to a to-space, live objects slid to one end

Three Performance Objectives

TODO: Space efficiency, fast collection, mutator performance. Each canonical collector achieves only two of the three.

TikZ diagram: triangle of the three objectives, with MS, SS, MC placed on the edge of the two objectives each one achieves

Locality and Contiguous Allocation

TODO: Bump-pointer allocation places contemporaneously allocated objects next to each other, giving better cache locality than a free list.

TikZ diagram: bump-pointer allocation (cursor moving right in a contiguous region) vs free-list allocation (objects scattered into holes)

The Space-Time Trade-off

TODO: Explain the crossing lines: MS wins in small heaps, SS wins in large heaps thanks to 10% better mutator time, MC is uncompetitive.

Copy from the paper: Figure 1 (performance trade-offs of MS, SS, MC)

Related Work

TODO: Earlier mark-region collectors (JRockit, IBM thread-local heaps), Metronome's on-demand defragmentation, Garbage-First, generational collectors.

Mark-Region Collection

Mark-Region Collectors

TODO: Sweep-to-region: memory is split into regions, a region with any live object is unavailable, the rest are bump allocated into.

TikZ diagram: naive mark-region heap: fixed regions, each either free or unavailable because it holds a live object

Two Open Questions

TODO: (1) How big should regions be? Large regions waste space, small ones slow collection. (2) How to defragment, since one live object pins a whole region?

Blocks and Lines

TODO: Immix manages memory at two granularities: coarse 32KB blocks and fine 128B lines. Objects may span lines but not blocks.

TikZ diagram: heap → 32KB blocks → 256 lines of 128B each, with small objects inside lines

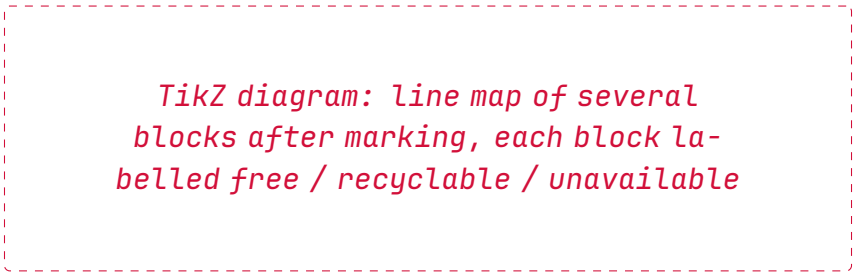
Initial Allocation

TODO: All blocks start empty. A thread-local allocator takes an empty block from the global allocator and bump allocates into it until the heap is exhausted, which triggers a collection.

TikZ diagram: global free block pool handing empty blocks to a thread-local allocator, which fills each block with a bump pointer

Identification and Reclamation

TODO: Tracing marks objects and their lines in a line map. A coarse-grained sweep classifies blocks as free, recyclable, or unavailable.



Steady State Allocation

TODO: Threads allocate into recycled blocks in address order: scan the line map for a hole, bump allocate into it, move to the next hole. Empty blocks are used only after all recyclable blocks.

TikZ diagram: recycled block with marked and free lines, cursor and limit pointing to the current hole, then jumping to the next hole

Immix Heap Organization

TODO: Walk through the paper's figure: global free block allocator, recycled blocks, cursor and limit, line mark legend.

Copy from the paper: Figure 2 (immix heap organization)

Recycling and Allocation Policies

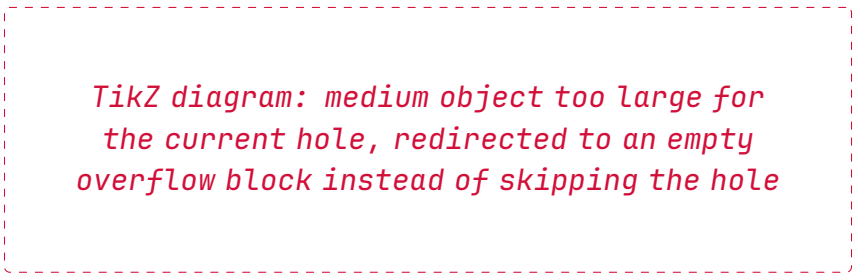
TODO: A block with at least $F = 1$ free line is recyclable. Recyclable blocks are used in address order, free blocks last because the large object space also needs them.

Parallelism

TODO: Synchronized global block allocator, unsynchronized thread-local allocators. Racy marking is safe; line marks are bytes to avoid synchronization.

Demand-Driven Overflow Allocation

TODO: Medium objects (larger than a line) that do not fit the current hole go to a separate overflow allocator using empty blocks, so small holes are not skipped.



Opportunistic Defragmentation

Why Defragment?

TODO: A pure mark-region collector never moves objects, so fragmented blocks stay partially used forever.

Opportunistic Evacuation

TODO: Before collection, choose candidate blocks to evacuate and target space. During marking, move objects off candidates and leave forwarding pointers; stop moving when space runs out. 2.5% of the heap is reserved as evacuation headroom.

TikZ diagram: fragmented candidate block, live objects copied into holes of a target block, forwarding pointers left behind

Candidate Selection

TODO: Defragment on demand. Mark and available histograms indexed by hole count; select the most fragmented blocks that fit the available space.

TikZ diagram: mark and available histograms by hole count, with a cut-off line where required space exceeds available space

Mixing Marking and Evacuation

TODO: Per-object decision during the trace: unmarked and unmovable → mark; unmarked and movable → evacuate and forward; forwarded → update reference. Single pass.

TikZ diagram: flowchart of the per-object decision during tracing

Parallel Evacuation and Pinning

TODO: Compare-and-swap to race on forwarding, paid only in the specialized defragmenting trace. Pinned objects are simply marked in place.

Implementation Details

Block and Line Size

TODO: 128B lines roughly match cache lines and fit 1-4 objects; 32KB blocks roughly match pages. Worst-case fragmentation bounds: about 25% per block, 97% per line in theory. Metadata is one byte per line and four per block: 0.8% overhead.

Conservative Line Marking

TODO: Most objects are under 128B and span at most two lines. Mark only the start line; the allocator skips the first line of each hole, which implicitly covers the second line. Medium objects are marked exactly.

TikZ diagram: small object straddling two lines: only the first line is marked, the next line is treated as implicitly marked

Optimizing Hot Paths

TODO: The allocator matches semi-space speed and tracing comes close to mark-sweep. Lines are marked when an object is scanned, not when it is marked, which hides the cost.

Evaluation

Methodology

TODO: 20 DaCapo, SPECjvm98 and pjbb2000 benchmarks; Jikes RVM with MMTk; Core 2 Duo, AMD Athlon, PowerPC 970; replay compilation; heaps from 1× to 6× the minimum.

Full Heap Performance

TODO: Immix beats all canonical collectors at every heap size on all three architectures, matching SS mutator time and MS collection time.

Copy from the paper: Figure 3 (full heap performance comparison)

Full Heap Results at 2× Minimum Heap

TODO: Per-benchmark slice: immix geomean 0.93 vs MS, worst case 1.01, best 0.82 (jython).

Copy from the paper: Table 1 (full heap performance at 2× minimum heap)

Space Efficiency

TODO: Minimum heap size relative to mark-sweep: immix 86%, mark-compact 83%, semi-space far larger.

Copy from the paper: Figure 5 (minimum heap sizes)

Immix in a Generational Collector

TODO: Semi-space nursery with an immix mature space (G|SS-IX) vs the production collector G|SS-MS; slightly better on average, 5% better on pjbb2000.

TikZ diagram: generational heap: bump-allocated nursery, survivors promoted into the immix mature space

Generational Performance at 1.5× Minimum Heap

TODO: Per-benchmark slice normalized to G|SS-MS: G|SS-IX geomean 0.99, in-place G|IX-IX 1.02, G|MS-MS 1.10.

Copy from the paper: Table 2 (generational performance at 1.5× minimum heap)

In-Place Generational: Sticky Mark Bits

TODO: Mark bits stay set between collections, so a nursery collection only traces newly allocated objects without moving anything. A write barrier remembers modified old objects. Supports pinning, unlike a copying nursery.

TikZ diagram: heap after a full collection with old objects marked; new unmarked objects allocated between them; a nursery collection traces only the unmarked ones

Generational Results

TODO: Copying generational, in-place (sticky mark bits) generational, and all immix variants compared.

Copy from the paper: Figure 6 (generational collector performance)

Selected Benchmarks

TODO: javac and luindex: full heap immix is the best collector. pjbb2000: immix variants beat the production collector.

Copy from the paper: Figure 4 (selected benchmarks)

What Makes Immix Work

TODO: Ablation: block-only marking inflates marked space by 93%, lines without defragmentation by 23%. Defragmentation rarely triggers but frees 95% of candidate space. Overflow allocation handles about 4% of allocation.

Conclusion

Conclusion

TODO: Mark-region is a new collector family; opportunistic defragmentation mixes copying and marking in one pass; immix achieves all three performance objectives.

References

- Stephen M. Blackburn, Kathryn S. McKinley. *Immix: A Mark-Region Garbage Collector with Space Efficiency, Fast Collection, and Mutator Performance*. PLDI, 2008.